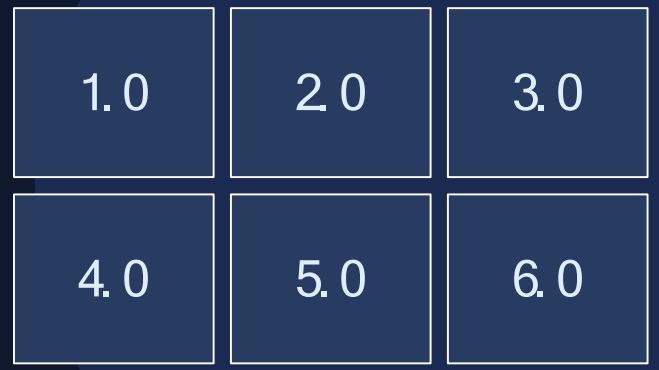
HANDS-ON LECTURE

PyTorch TENSORS

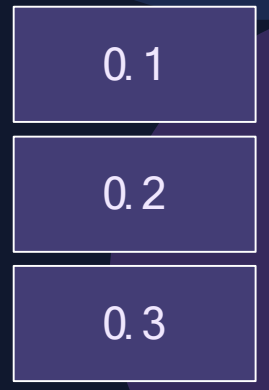
Shapes • Multiplication • Dot Products
Flatten & Reshape • Learnable Weights

EXAMPLES + VISUALS + CLASSROOM MCQS

Python • Google Colab • Neural Networks



shape = (2, 3)



Today: five skills for building neural networks

Every concept is demonstrated with simple PyTorch code.

01

Create tensors

Numbers, vectors, and matrices

02

Read shapes

Rows, columns, dimensions

03

Multiply tensors

Element-wise, dot, matrix

04

Transform shapes

Flatten and reshape

05

Train with weights

Float values and gradients

> Key question: how does a collection of numbers become a prediction?

A tensor can have 0, 1, 2, or more dimensions

Dimensions describe how values are organized—not how large the values are.

Scalar • 0D

One value

5.0

shape = ()

Vector • 1D

One axis

1	2	3

shape = (3,)

Matrix • 2D

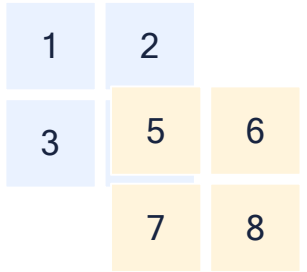
Rows and columns

1	2	3
4	5	6

shape = (2, 3)

3D tensor

Stack of matrices

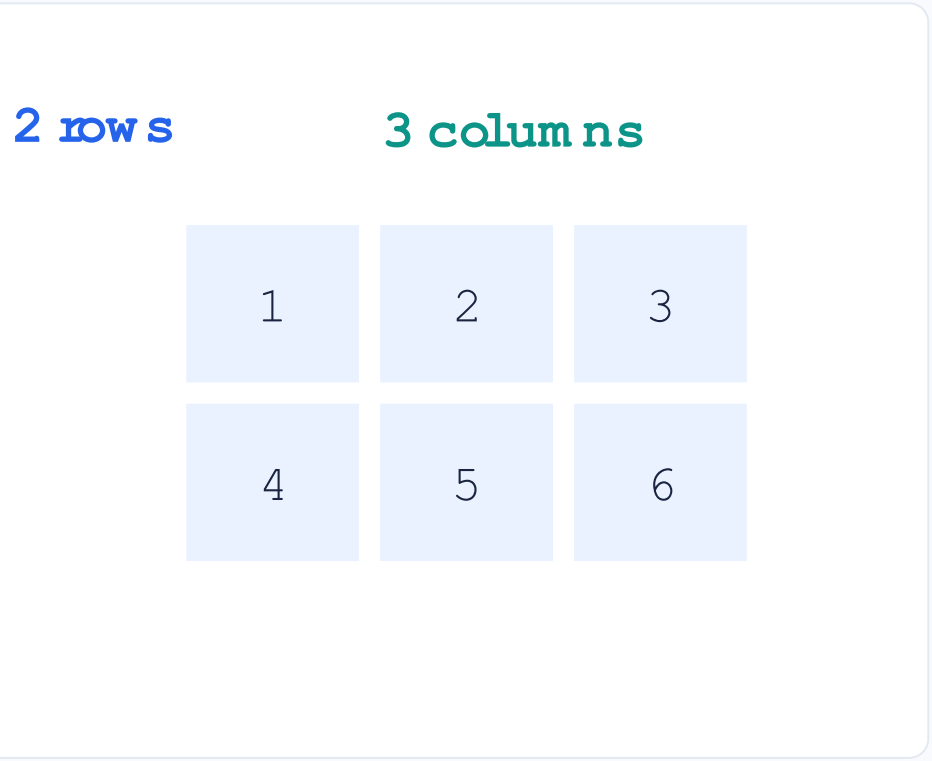


shape = (2, 2, 2)

In code: `torch.tensor(5.0)` • `torch.tensor([1, 2, 3])` • `torch.tensor([[1, 2], [3, 4]])`

Read the shape of a matrix

A 2 × 3 tensor has 2 rows, 3 columns, and 6 total elements.



PYTHON / PYTORCH

```
x = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])

print(x.shape)   # (2, 3)
print(x.ndim)    # 2 dimensions
print(x.numel()) # 6 elements
```

> shape = (rows, columns) for a 2D tensor

Tensor data types: integers vs decimals

The dtype determines how a number is stored and what computations are supported.

INTEGER VALUES

[1, 2, 3]

Common dtype: torch.int64

Counts, labels, indices

FLOAT VALUES

[1.0, 2.0, 3.0]

Common dtype: torch.float32

Features, weights, gradients

torch.tensor([1, 2]) creates an integer tensor; torch.tensor([1., 2.]) creates a float tensor by default.

Element-wise multiplication: the * operator

Multiply corresponding entries. For equal-shaped vectors, the shape stays the same.

$$1 \times 4 = 4$$

$$2 \times 5 = 10$$

$$3 \times 6 = 18$$

PYTHON / PYTORCH

```
A = torch.tensor([1., 2., 3.])
B = torch.tensor([4., 5., 6.])
print(A * B)  # tensor([4., 10., 18.])
```

The same * operator also works with matrices

Each position is multiplied with the matching position.

Matrix A		×	Matrix B		=	Result: A * B	
1	2		5	6		5	12
3	4		7	8		21	32

> No rows are summed here! This is not matrix multiplication.

Dot product: multiply, then add

torch.dot requires two 1D tensors with the same number of elements.

1 × 4	+	2 × 5	+	3 × 6
= 4		= 10		= 18

4 + 10 + 18 = 32

PYTHON / PYTORCH

```
A = torch.tensor([1., 2., 3.])
B = torch.tensor([4., 5., 6.])
print(torch.dot(A, B))  # tensor(32.)
```

A neuron is a weighted dot product + bias

Three inputs • three weights • one output

input 2.0	×	weight 0.1	→	0.2
input 3.0	×	weight 0.2	→	0.6
input 4.0	×	weight 0.3	→	1.2

Sum

0.2 + 0.6 + 1.2

+ bias 0.5

PREDICTION = 2.5

PYTHON / PYTORCH

```
x = torch.tensor([2., 3., 4.])
w = torch.tensor([0.1, 0.2, 0.3])
b = torch.tensor(0.5)
prediction = torch.dot(x, w) + b
```

When should I use * versus dot or @?

The same two vectors produce different kinds of output.

ELEMENT-WISE *

[1, 2, 3] * [4, 5, 6]

→ [4, 10, 18]

Output is a vector

DOT PRODUCT

dot([1,2,3], [4,5,6])

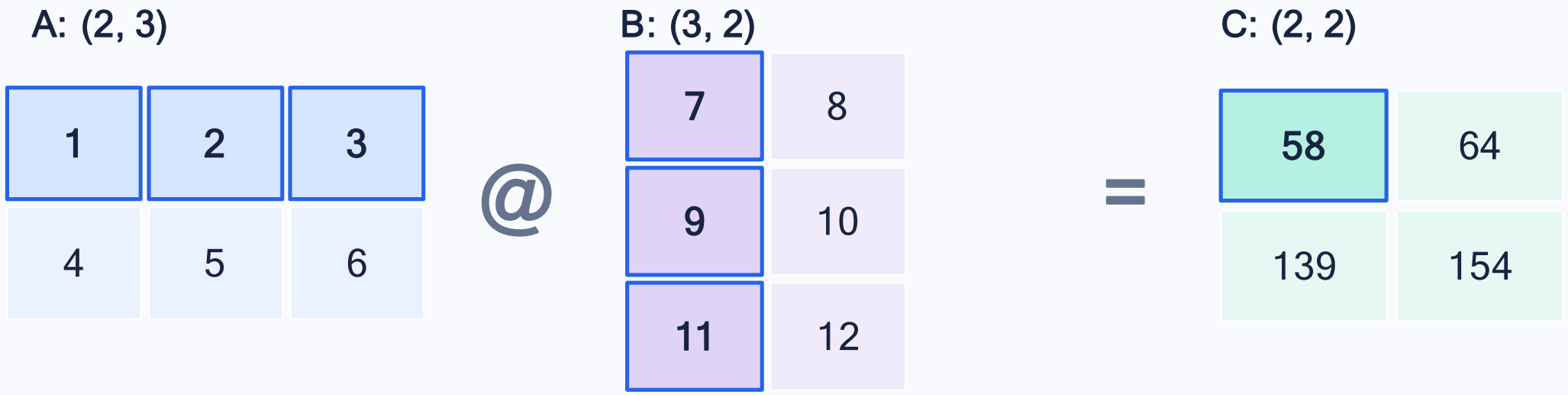
→ 32

Output is one number (scalar)

For two 1D vectors: A @ B is also a dot product. For 2D matrices: A @ B is matrix multiplication.

Matrix multiplication: row × column

A (2 × 3) matrix @ a (3 × 2) matrix gives a (2 × 2) result.



Top-left entry = (1 × 7) + (2 × 9) + (3 × 11) = 58

```
C = A @ B    # same as torch.matmul(A, B)   → shape (2, 2)
```

Matrix multiplication: the shape rule

The two inner dimensions must be equal.

VALID

(2, 3) @ (3, 4)

$3 = 3$

Output shape: (2, 4)

INVALID

(2, 3) @ (2, 4)

$3 \neq 2$

RuntimeError: incompatible shapes

> Rule: (m, n) @ (n, p) → (m, p)

Flatten: turn a matrix into one vector

The numbers and their order are unchanged. Only the shape changes.

Original matrix

1	2	3
4	5	6

shape: (2, 3)



1D vector

1	2	3	4	5	6

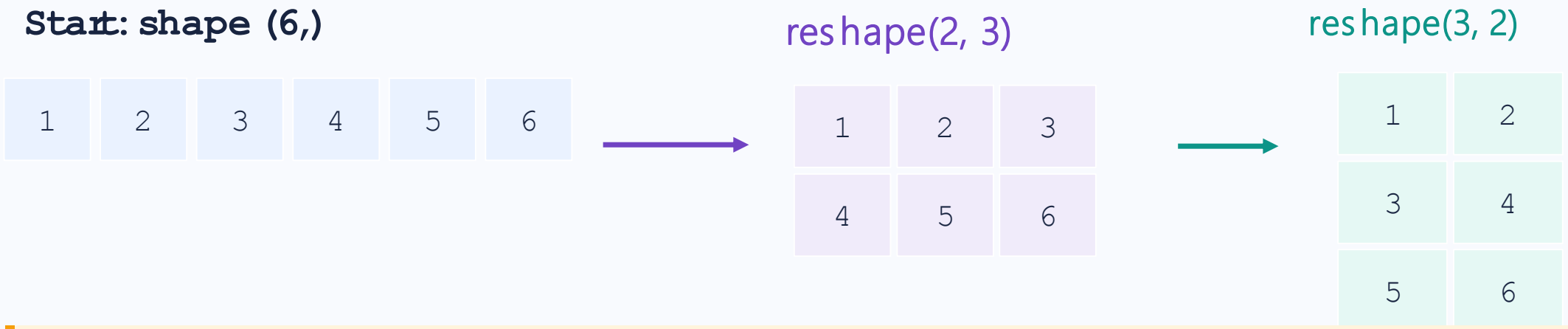
shape: (6,)

PYTHON / PYTORCH

```
flat = torch.flatten(x)
print(flat)       # tensor([1.,2.,3.,4.,5.,6.])
print(flat.shape) # torch.Size([6])
```

Reshape: choose a new arrangement

The total number of elements must stay the same.



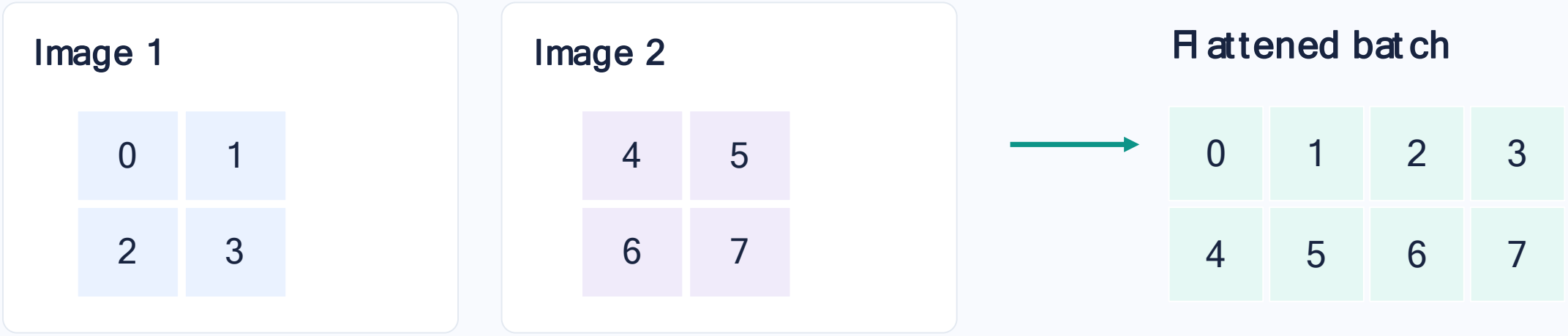
Reshape does not automatically perform matrix multiplication, and it does not transpose a matrix.

PYTHON / PYTORCH

```
x = torch.tensor([1., 2., 3., 4., 5., 6.])
print(x.reshape(2, 3))   # 2 rows, 3 cols
print(x.reshape(3, 2))   # 3 rows, 2 cols
```

Flatten image features—but keep the batch

A batch dimension tells PyTorch how many examples you are processing.



Before: (2, 2, 2)

After: (2, 4)

PYTHON / PYTORCH

```
images = torch.arange(8.).reshape(2, 2, 2)
features = torch.flatten(images, start_dim=1)
print(features.shape)  # torch.Size([2, 4])
```

Why matrix multiplication powers dense layers

One operation produces outputs for multiple examples and neurons.

Batch of inputs				Weights			Predictions	
1	2	3	@	0.1	0.4	=	1.4	3.2
4	5	6		0.2	0.5		3.2	7.7
				0.3	0.6			

2 examples × 3 features

3 features × 2 neurons

2 examples × 2 outputs

PYTHON / PYTORCH

```
X = torch.tensor([[1.,2.,3.], [4.,5.,6.]])
W = torch.tensor([[.1,.4], [.2,.5], [.3,.6]])
Y = X @ W   # shape (2, 2)
```

Why neural network weights should be floating-point

Gradient descent improves a weight using small decimal updates.

Update rule

$$\text{new weight} = \text{old weight} - \text{learning rate} \times \text{gradient}$$

2.000 Initial → 1.970 Update 1 → 1.940 Update 2 → 1.910 Update 3

With lr = 0.1 and gradient = 0.3: 2.0 − 0.1 × 0.3 = 1.97. An integer-only weight cannot represent 1.97.

Note: Quantized integer weights can be useful at inference time; standard gradient-based training uses floating-point weights.

Autograd: PyTorch calculates derivatives for us

Set requires_grad=True, compute a result, then call backward().



Derivative: d(w²)/dw = 2w. At w = 2, the gradient is 4.

PYTHON / PYTORCH

```
w = torch.tensor(2.0, requires_grad=True)
result = w ** 2
result.backward()
print(w.grad.item())    # 4.0
```

One neuron: from prediction to gradients

The loss tells us how far the prediction is from the target.

Prediction	Target	Squared loss	Weight gradients
2.5	2.0	0.25	[2, 3, 4]

PYTHON / PYTORCH

```
x = torch.tensor([2., 3., 4.])
w = torch.tensor([.1, .2, .3], requires_grad=True)
b = torch.tensor(0.5, requires_grad=True)

prediction = torch.dot(x, w) + b    # 2.5
loss  = (prediction - 2.0) ** 2     # 0.25
loss.backward()
print(w.grad)  # tensor([2., 3., 4.])
```

Gradients calculate direction; an optimizer changes weights

backward() does not change weights by itself. optimizer.step() does.

1 **Predict**

forward pass

→

2 **Compare**

loss

→

3 **Differentiate**

backward()

→

4 **Update**

step()

PYTHON / PYTORCH

```
optimizer = torch.optim.SGD([w, b], lr=0.1)

optimizer.zero_grad()  # clear old gradients
prediction = torch.dot(x, w) + b
loss = (prediction - 2.0)**2
loss.backward()      # compute gradients
optimizer.step()     # update weights
```

Common mistakes—and simple fixes

Most early errors come from dtype mismatches or incompatible shapes.

Integer gradient tensor	torch.tensor(2, requires_grad=True)	Use 2.0 or dtype=torch.float32
Wrong matmul dimensions	(2, 3) @ (2, 4)	Inner dimensions must match
Mixing dot and element-wise	torch.dot(A, B) gives a scalar	Use A * B to keep element products

Always print x.shape and x.dtype when debugging.

Quick check: tensors and multiplication

Ask students to choose the answer before showing the next slide.

Q1. What is the shape of `[[1,2,3],[4,5,6]]`?

A. (6,)

B. (3, 2)

C. (2, 3)

Q2. What is `torch.dot([1,2], [3,4])`?

A. [3, 8]

B. 11

C. [4, 6]

Quick check: shapes and training

Think about operations and why weights are represented using floating-point values.

Q3. What is the result of flattening a (2, 3) tensor?

A. (3, 2)

B. (6,)

C. (2, 3, 1)

Q4. Why do we use floating-point training weights?

A. To allow small gradient updates

B. To avoid addition

C. To store only whole numbers

Answers & the essential takeaways

The goal is to choose the correct operation and shape before training a model.

MCQ answer key

Q1 C	2 rows, 3 columns
Q2 B	$1 \times 3 + 2 \times 4 = 11$
Q3 B	6 values → (6,)
Q4 A	small decimal updates

Remember these five rules

- shape tells how values are organized
- * multiplies matching entries
- dot multiplies and sums
- flatten / reshape reorganize data
- float weights enable gradient updates

Practice: predict each output shape before running your code in Colab.